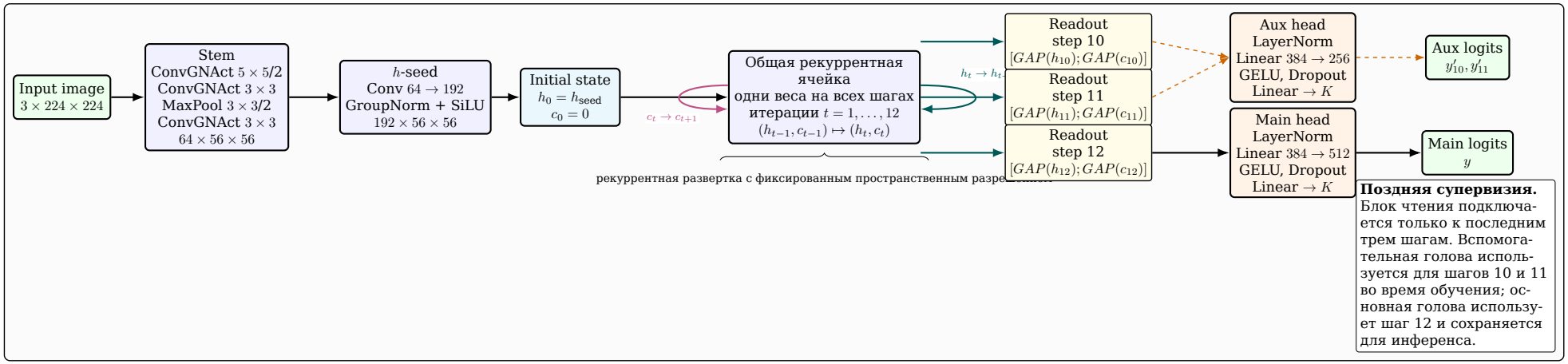
GRLNet-StabhRec: single-layer recurrent architecture



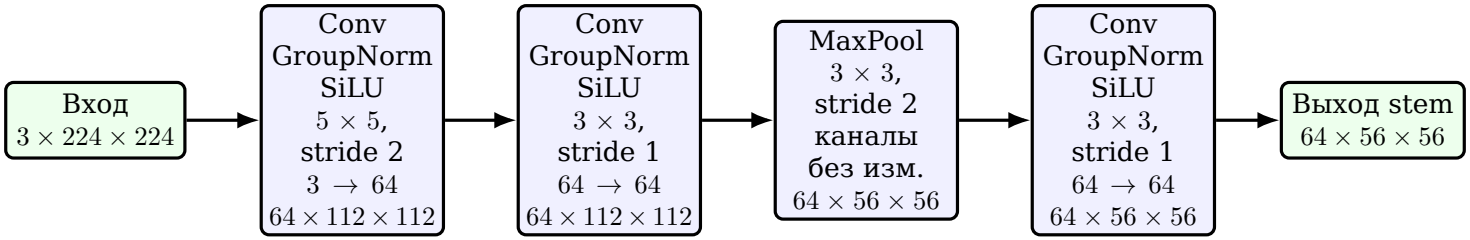
Текстовое описание общей схемы. Входное изображение размера $3 \times 224 \times 224$ сначала обрабатывается блоком *stem*, который уменьшает пространственное разрешение до 56×56 и формирует карту признаков размера $64 \times 56 \times 56$. Затем блок h-seed отображает этот тензор в скрытое рекуррентное пространство, создавая начальное состояние $h_0 \in \mathbb{R}^{192 \times 56 \times 56}$, тогда как состояние памяти инициализируется как $c_0 = 0$.

Ядром модели является одна общая рекуррентная ячейка, применяемая в течение 12 последовательных шагов. Одни и те же параметры ячейки используются на каждом шаге, поэтому сеть увеличивает эффективную глубину за счет итеративного уточнения, а не за счет стека различных слоев. В течение всей рекуррентной развертки и скрытое состояние, и состояние памяти остаются на одной и той же пространственной сетке.

Предсказания снимаются не с каждого рекуррентного шага. Вместо этого блоки readout подключаются только к последним трем шагам, а именно к шагам 10, 11 и 12. На каждом из этих шагов модель выполняет глобальный average pooling как для h_t, так и для c_t, конкатенирует полученные векторы и использует итоговое представление как вход классификатора.

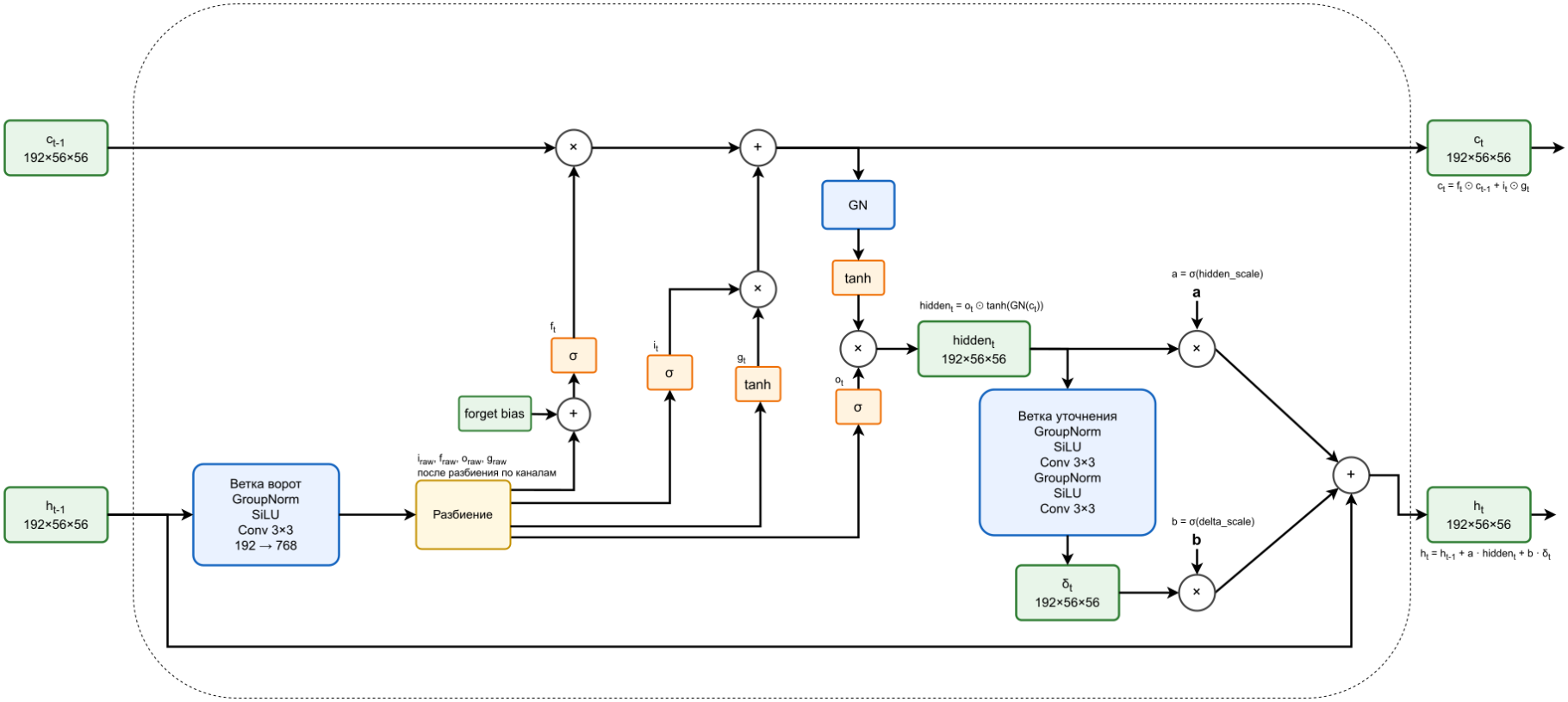
Финальная предиктовая голова работает с readout шага 12 и формирует logits для инференса. Во время обучения дополнительная auxiliary-head применяется также к readout шагов 10 и 11, обеспечивая позднюю дополнительную супервизию. Таким образом, при инференсе используется только последнее рекуррентное состояние, тогда как обучение получает дополнительный сигнал ошибки на двух предшествующих поздних шагах.

Отдельная схема Stem.



Краткое описание блока Stem. Первый сверточный слой с ядром 5×5 и шагом 2 выполняет начальное извлечение локальных признаков и одновременно уменьшает пространственное разрешение входа. Второй сверточный слой 3×3 уточняет полученные низкоуровневые признаки без изменения числа каналов и пространственного масштаба. Затем операция max-pooling дополнительно уменьшает разрешение, сохраняя наиболее выраженные локальные ответы. Последний сверточный слой 3×3 формирует итоговое представление блока *stem* размера $64 \times 56 \times 56$, которое затем подается на вход блока h-seed для инициализации рекуррентного скрытого состояния.

Отдельная схема рекуррентной ячейки.



Подробное описание этапов обработки в рекуррентной ячейке. На вход ячейки поступают два состояния предыдущего шага: скрытое состояние h_{t-1} и состояние памяти c_{t-1}. Сначала из h_{t-1} через ветку *gate branch* последовательно вычисляются нормализованные и нелинейно преобразованные признаки, после чего свертка формирует тензор из четырех групп каналов. Этот тензор разбивается на сырые карты i_{raw}, f_{raw}, o_{raw} и g_{raw}, соответствующие входным воротам, воротам забывания, выходным воротам и кандидату обновления памяти.

Далее к ветви забывания добавляется константный *forget bias*, после чего применяются нелинейности: сигмоида для i_t, f_t и o_t, а гиперболический тангенс для g_t. Состояние памяти обновляется по LSTM-подобному правилу

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t,$$

где одна часть старой памяти сохраняется через f_t, а новая информация записывается через i_t и g_t.

После этого новое состояние памяти c_t нормализуется, проходит через функцию tanh и затем поэлементно умножается на выходные ворота o_t. Так формируется основной кандидат обновления скрытого состояния

$$hidden_t = o_t \odot \tanh(GN(c_t)).$$

Этот сигнал отражает основной результат текущего рекуррентного шага.

Затем $hidden_t$ подается в дополнительную ветку *delta / refine*, состоящую из двух последовательных блоков *GroupNorm* $\rightarrow$ *SiLU* $\rightarrow$ *Conv*. Эта ветка вычисляет корректирующую добавку δ_t, предназначенную для более тонкого нелинейного уточнения скрытого состояния. Таким образом, основная ветвь задает базовое gated-обновление, а refine-ветка формирует дополнительную поправку к нему.

На последнем этапе старое скрытое состояние h_{t-1} передается по residual-связи напрямую к выходу ячейки. К нему добавляются два новых вклада: основной сигнал $hidden_t$ и корректирующий сигнал δ_t, каждый со своим обучаемым скалярным коэффициентом:

$$h_t = h_{t-1} + a \cdot hidden_t + b \cdot \delta_t, \qquad a = \sigma(hidden_scale), \quad b = \sigma(delta_scale).$$

Коэффициенты a и b являются обучаемыми глобальными весами, которые регулируют, насколько сильно в итоговое скрытое состояние должны входить основной gated-update и refine-поправка. В результате выходами ячейки становятся обновленные состояния c_t и h_t, сохраняющие ту же пространственную структуру, что и на входе.